

CS112: Wednesday 16 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Monday we read **parameter lists** and watched `const` refuse an assignment, after going back over in, out and in/out properly.

Today: a file is a stream, exactly like `cout`. The same `<<` and `>>` you have used since week 0 work on a file, so nothing about reading a file is new except opening it and noticing when it ends. And because opening can fail, the second half is what a function does when it cannot do its job at all.

TIME	BEAT
12:20	1. Streams, console and file side by side
12:38	2. Reading, and the end-of-file trap
12:52	3. When a function cannot do its job
13:10	4. Demo

Beat 1. Streams, console and file side by side (18 min)

```
ifstream fin("data.txt");           // in, from a file
ofstream fout("out.txt");            // out, to a file

fout << "hello" << endl;            // the same << as cout
```

Then the line that has to be there and that everyone forgets:

```
if (!fin) { /* it did not open */ }
```

A file that is not there does not announce itself. The stream just quietly fails, and every read after that does nothing.

Beat 2. Reading, and the end-of-file trap (14 min)

`>>` reads one whitespace-separated word. `getline` reads the rest of the line. Choosing between them is choosing what a unit of data is.

```
while (!fin.eof()) { fin >> word; cout << word << endl; }    //  
wrong  
while (fin >> word) { cout << word << endl; }                //  
right
```

Talk to your neighbor. The first loop prints the last word twice.
Why?

`eof()` reports on the read that already happened. It becomes true only after a read has failed, so the loop runs one extra time on a word it never got. The second version tests the read itself, which is the thing you actually care about.

Beat 3. When a function cannot do its job (18 min)

A function that returns an error code depends on the caller checking it, and the caller is free not to.

```
if (!fin) throw runtime_error("could not open the file");
```

`throw` abandons the function. Control unwinds outward until somebody catches it, and if nobody does, the program stops. Catch by reference, so the exception is not copied on its way out.

Beat 4. Demo (10 min)

Open a file that does not exist. Watch the throw leave one function, pass through another that does not catch it, and land in `main`.